Tervezési minták III.

1.rész

Halmaz reprezentálása

Gregorics Tibor

gt@inf.elte.hu

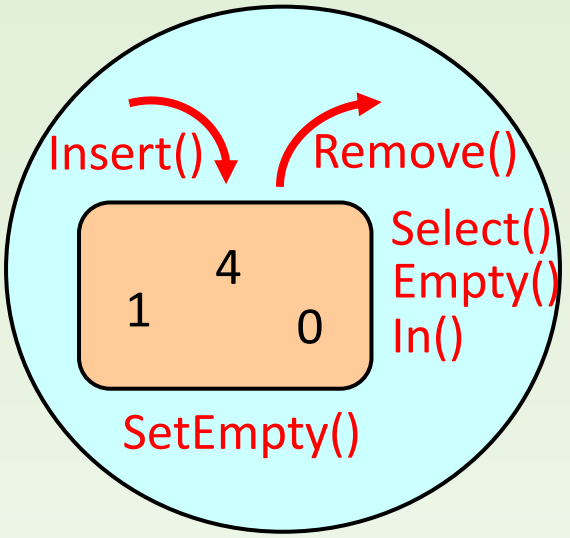
http://people.inf.elte.hu/gt/oep

Cél

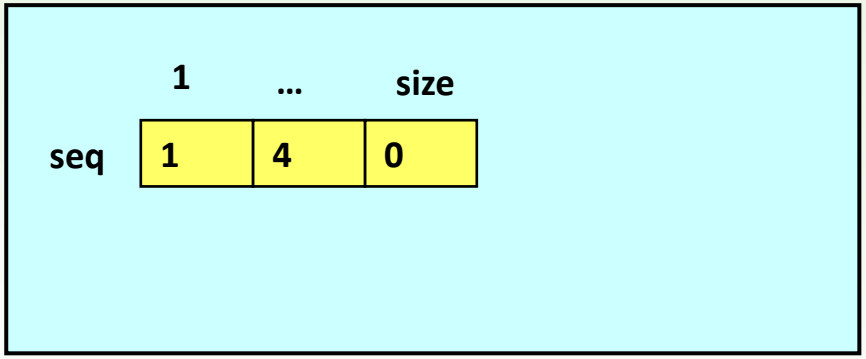
Készítsünk olyan kódot, amely segítségével természetes számokat tároló halmazok (gyűjtemények) típusa adható meg.

- Egy halmaz reprezentációját aszerint választhassuk ki, hogy van-e előre ismert felső korlátja a halmazban tárolandó természetes számoknak.
  - Ha nincs ilyen felső korlát: a halmaz elemeit egy sorozat tárolja.
  - Ha van ilyen felső korlát (max): a halmazt egy max+1 hosszú logikai értékeket tartalmazó tömb reprezentálja: a halmazban elvileg elhelyezhető számok a tömb indexei, és a tömb csak azon indexeknél tároljon igaz értéket, amelyek indexek – mint természetes számok – a halmazban vannak.
- Szeretnénk a reprezentációt elrejteni a halmazt használók elől: egy halmaz létrehozásánál csak azt kérdezzük meg, hogy tud-e a felhasználó előre felső korlátot mondani a halmazba kerülő természetes számokra, de ne kelljen tudnia arról, hogy ehhez milyen reprezentációt használunk.

Kétféle reprezentáció



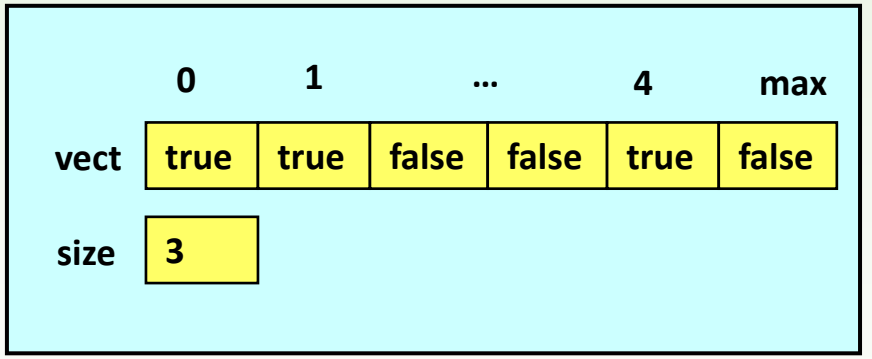
Sorozat



1. Dinamikusan változó hosszúságú sorozat.

2. Az Insert(), Remove(), In() számítási bonyolultsága lineáris, az Empty(), SetEmpty(), Select() konstans idejű.

Tömb



1. Rögzített méretű tömb és külön a halmazbeli elemek száma.

2. Az Insert(), Remove(), In(), Empty() számítási bonyolultsága konstans, a Select() és a SetEmpty() lineáris.

Halmaz típus sorozattal

$set(\mathbb{N})$

Típus-specifikáció

típusértékek

Olyan véges elemű halmazok, amelynek elemei természetes számok.

műveletek

üresé teszi a halmazt (SetEmpty)
$h{:=}\emptyset$ $h{:}set(\mathbb{N})$

betesz egy elemet a halmazba (Insert)
$h{:=}h \cup \{e\}$ $h{:}set(\mathbb{N}), e{:}\mathbb{N}$

kivesz egy elemet a halmazból (Remove)
$h{:=}h - \{e\}$ $h{:}set(\mathbb{N}), e{:}\mathbb{N}$

kiválasztja a halmaz egy elemét (Select)
$e{:}\in h$ $h{:}set(\mathbb{N}), e{:}\mathbb{N}$

üres-e a halmaz (Empty)
$l{:=}\ h{=}\emptyset$ $h{:}set(\mathbb{N}), l{:}\mathbb{L}$

benne van-e egy elem a halmazban (In)
$l{:=}\ e{\in}h$ $h{:}set(\mathbb{N}), e{:}\mathbb{N}, l{:}\mathbb{L}$

reprezentáció

$seq : \mathbb{N}^*$

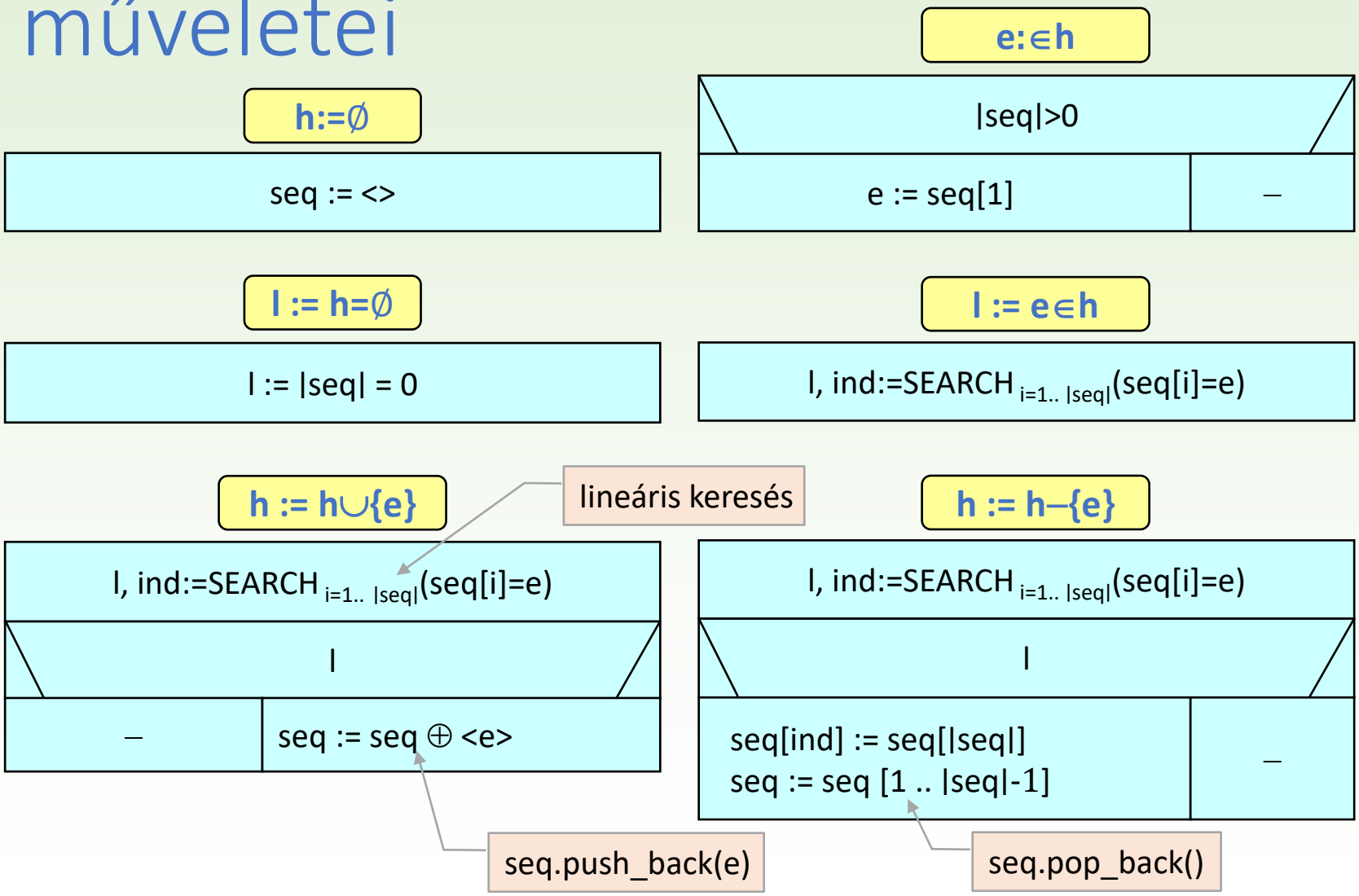
C++-ban: vector<int>

implementáció

műveletek programjai

Típus-megvalósítás

Sorozattal reprezentált halmaz műveletei



Halmaz típus tömbbel

set([0..max])

Típus-specifikáció

típusértékek

Olyan halmazok, amelynek elemei
0 és max közé eső természetes számok.

műveletek

üresé teszi a halmazt (SetEmpty)
$h:=\emptyset$ $h:set(\mathbb{N})$
betesz egy elemet a halmazba (Insert)
$h:=h\cup\{e\}$ $h:set(\mathbb{N}), e:\mathbb{N}$
kivesz egy elemet a halmazból (Remove)
$h:=h-\{e\}$ $h:set(\mathbb{N}), e:\mathbb{N}$
kiválasztja a halmaz egy elemét (Select)
$e:\in h$ $h:set(\mathbb{N}), e:\mathbb{N}$
üres-e a halmaz (Empty)
$l:= h=\emptyset$ $h:set(\mathbb{N}), l:\mathbb{L}$
benne van-e egy elem a halmazban (In)
$l:= e\in h$ $h:set(\mathbb{N}), e:\mathbb{N}, l:\mathbb{L}$

reprezentáció

$vect : \mathbb{L}^{0..max}$
$size : \mathbb{N}$

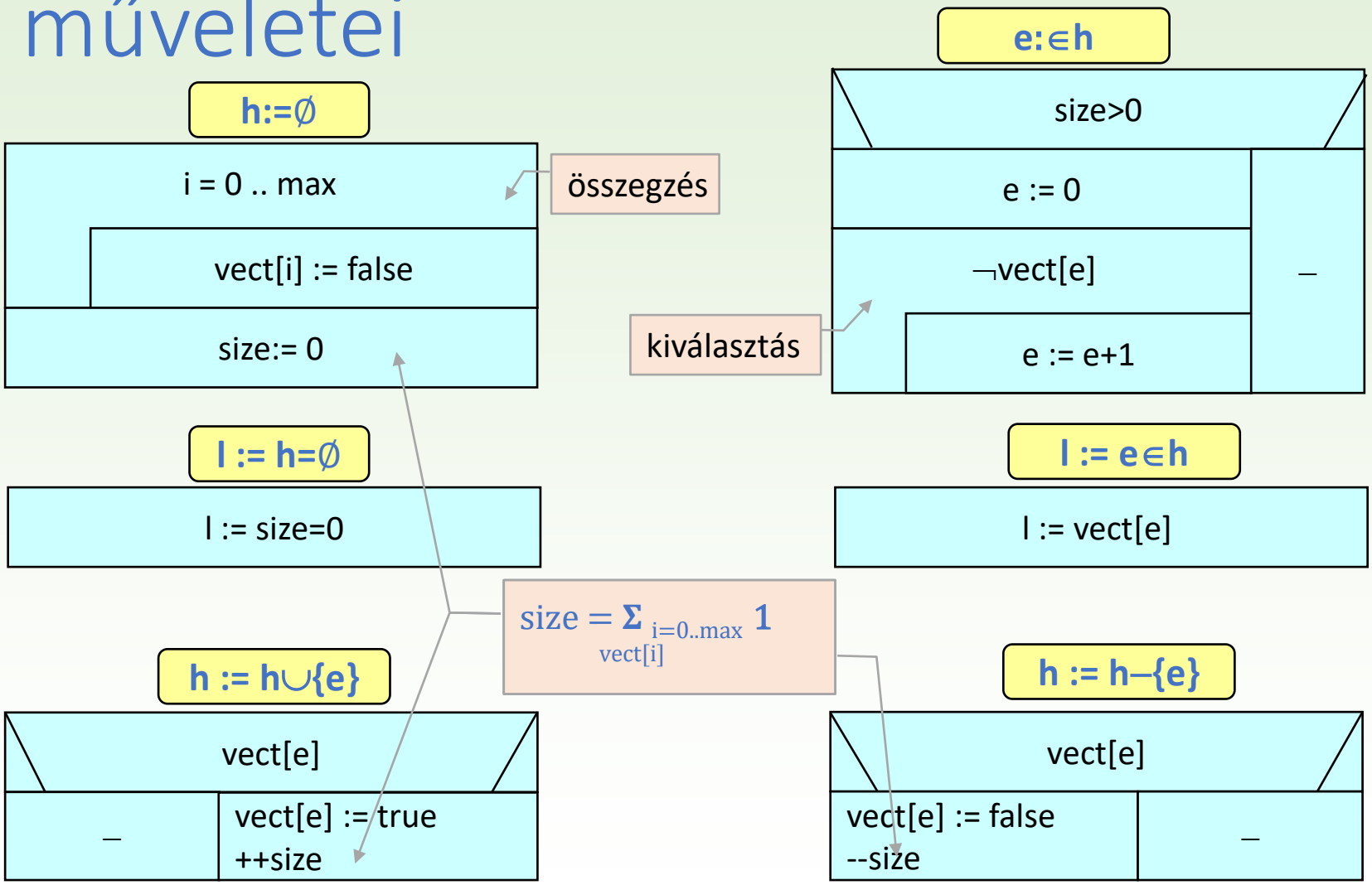
invariáns: $size = \sum_{i=0..max,\ vect[i]} 1$

implementáció

műveletek programjai

Típus-megvalósítás

Tömbbel reprezentált halmaz műveletei

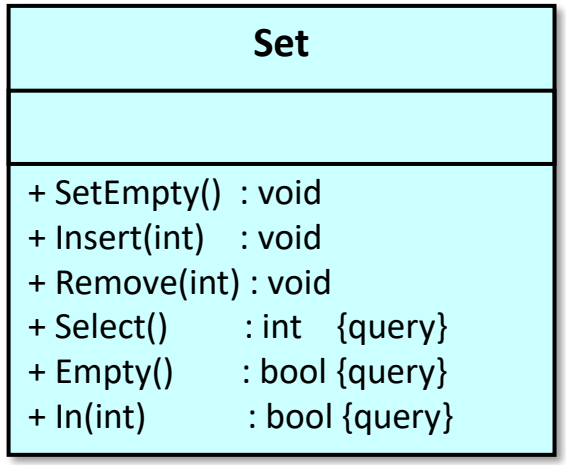


Halmaz osztály szolgáltatásai

```
class Set
{
      public:

            void setEmpty();
            void insert(int e);
            void remove(int e);
            int    select()  const;
            bool empty()  const;
            bool in(int e) const;

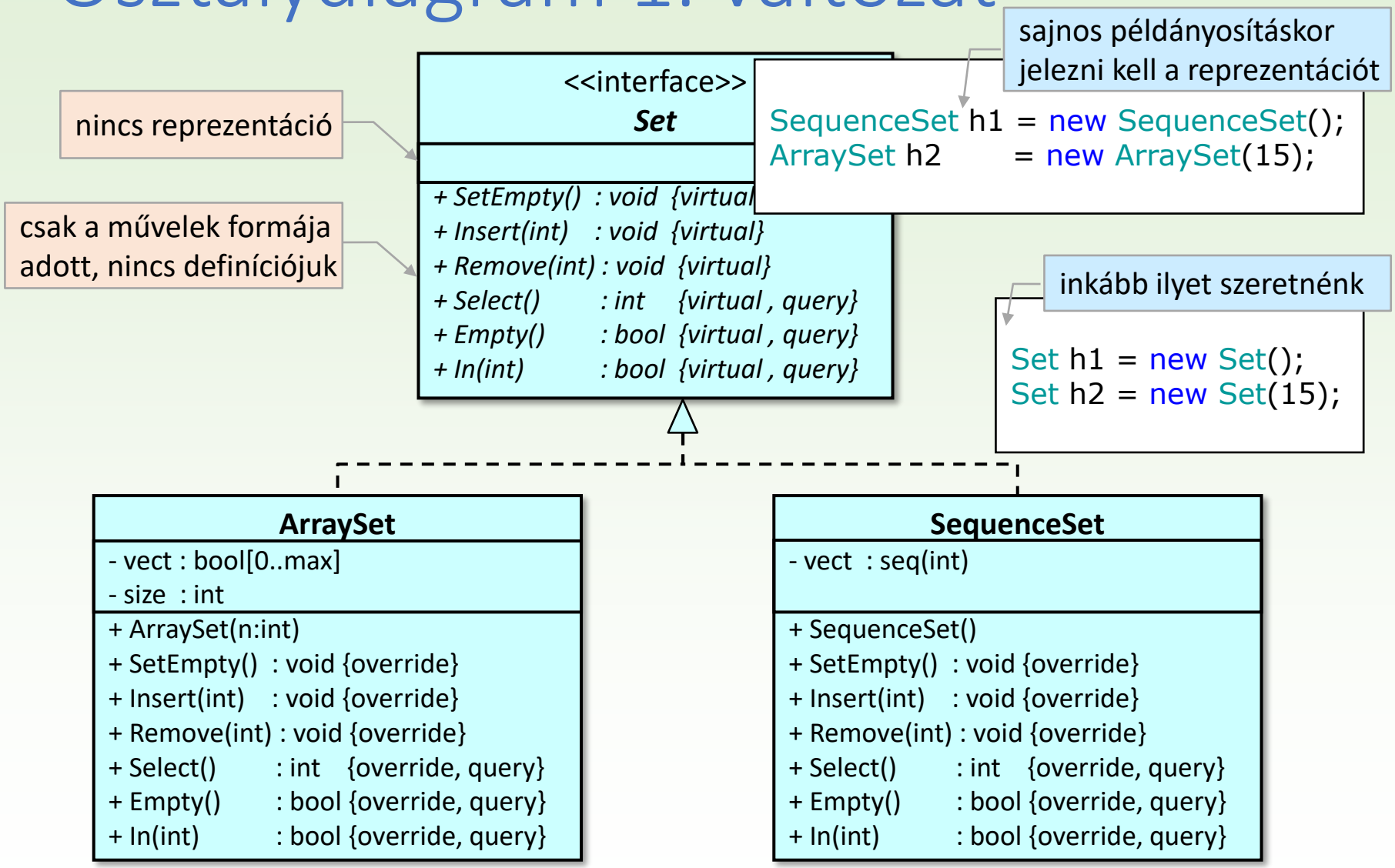
      private:
            ...
};
```



set.h

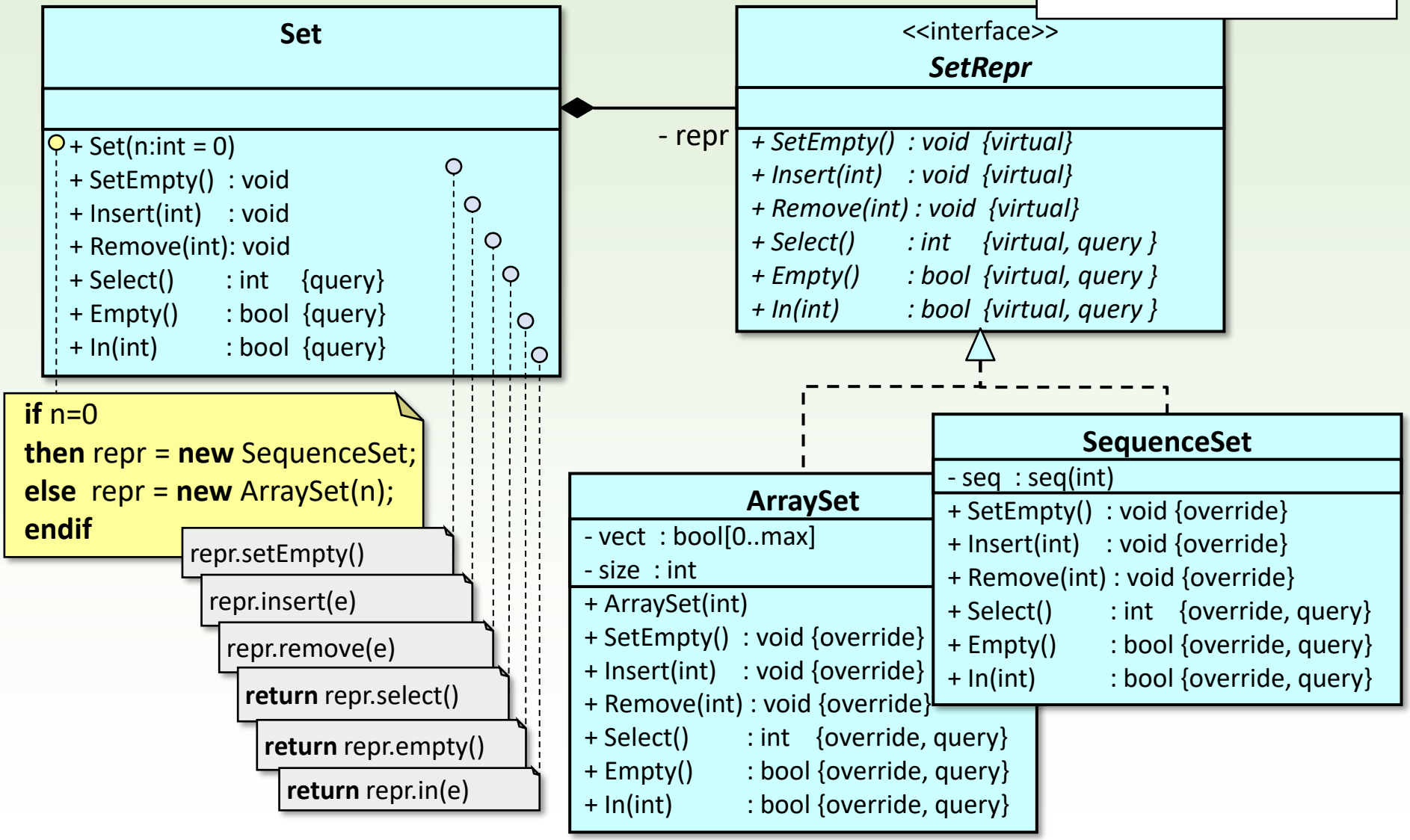
Hogyan írható le egyszerre mindkét reprezentáció?

Osztálydiagram 1. változat



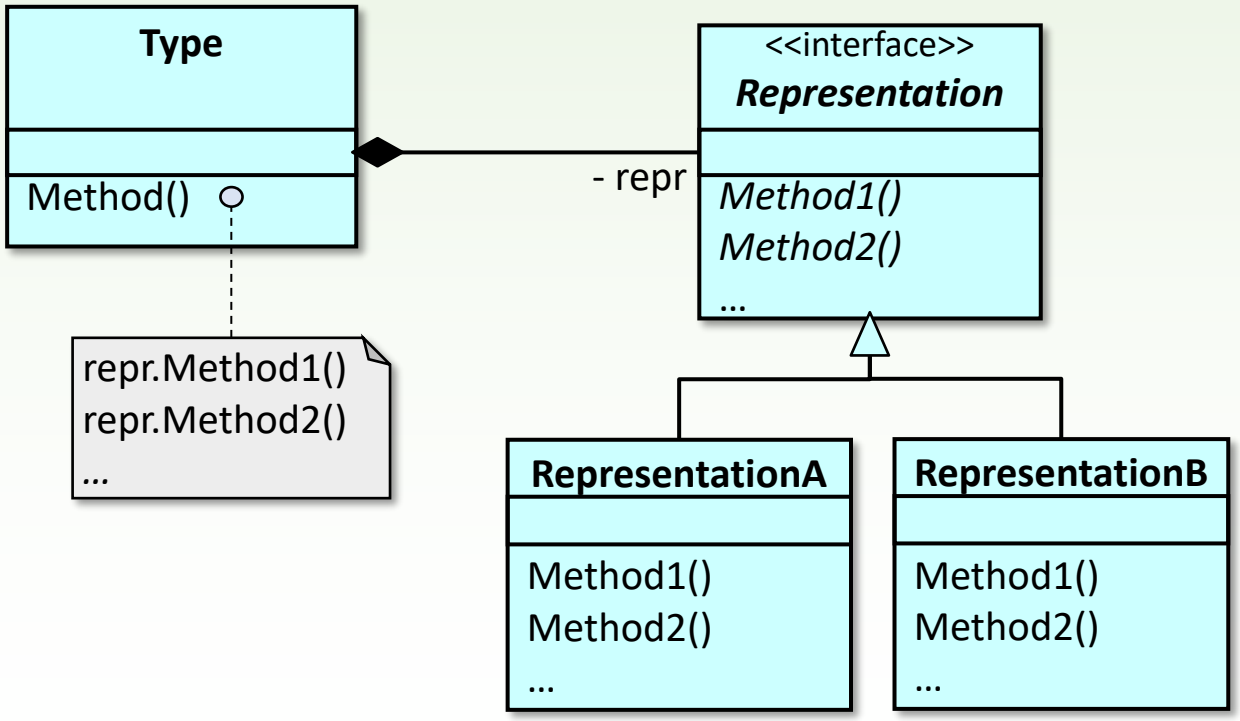
Osztálydiagram 2. változat

```
Set h1 = new Set();
Set h2 = new Set(15);
```



Híd (bridge) tervezési minta

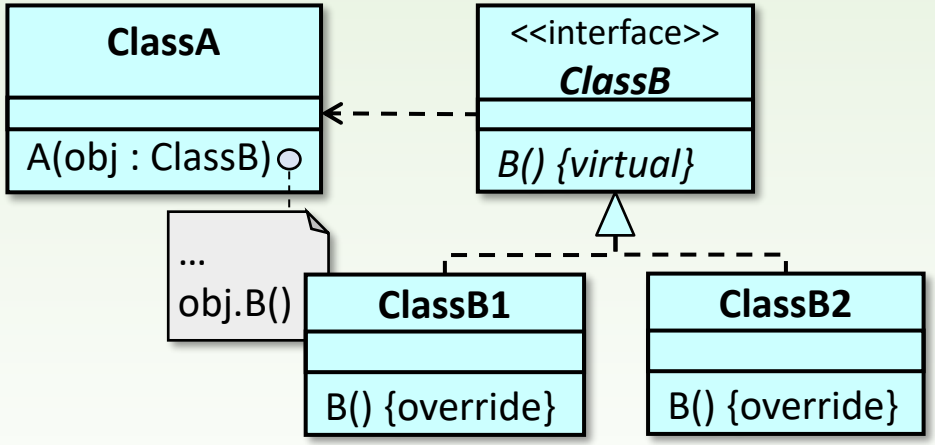
- Egy osztály reprezentációját leválasztjuk az osztályról azért, hogy azt rugalmasan, futási időben választhassuk ki.

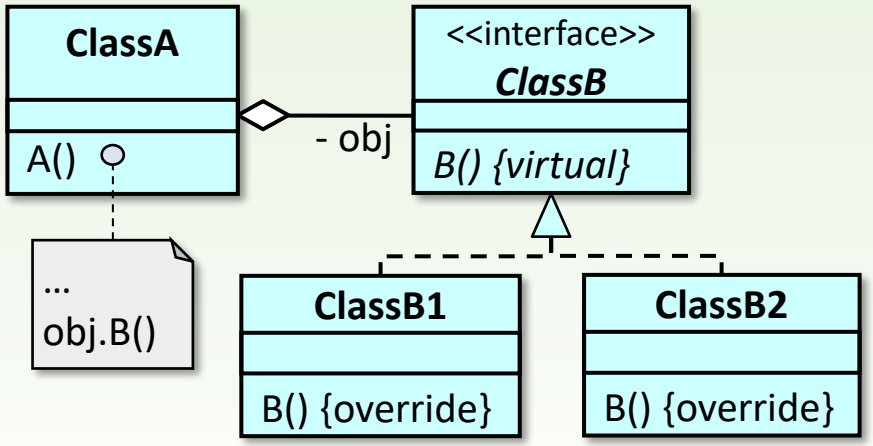


Tervezési minták hasonlósága

függőség vagy aggregáció

Stratégia: a *ClassB* jellemzően adattag nélküli osztályok interfésze, amelyek egyikének *B*() metódusát felhasználja a *ClassA* osztály *A*() metódusának működése.





Látogató: a *ClassA* osztály vagy alosztályának *A*() metódusa az obj típusától függően működik anélkül, hogy az *A*() metódusban ismerni kelljen a *ClassB* alosztályait.

Híd: a *ClassB* adattagokkal rendelkező osztályok interfésze, amelyek a *ClassA* osztállyal leírt típus-specifikációhoz adnak reprezentációt és implementációt.

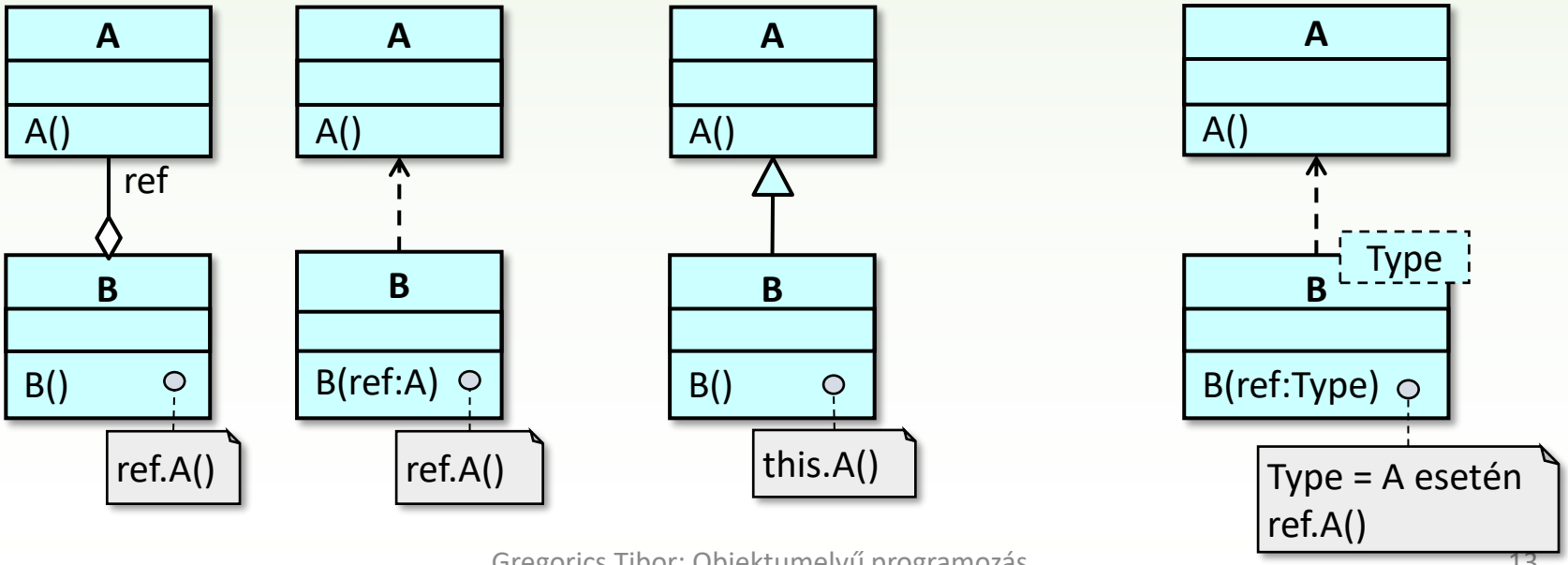
függőség

kompozíció

Felelősség átruházás

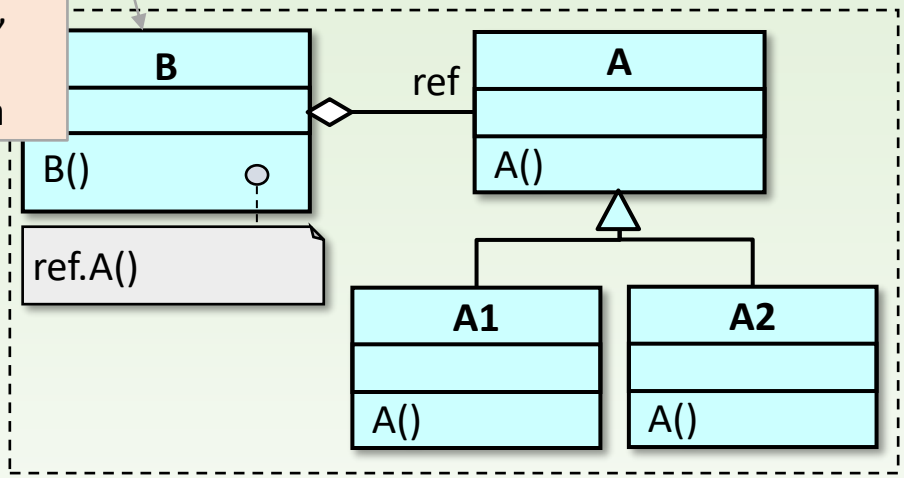
- A felelősség átruházás (*dependency injection*) egy objektum viselkedését (metódusainak működését) másik osztály kódjától teszi függővé.

- Objektum befecskendezéssel: az objektum metódusa egy másik objektum metódusát hívja.
- Származtatással: az objektum metódusa az ősosztályának nem felülírt metódusát hívja.
- Osztálysablonnal: az objektum metódusa a sablonparaméterében adott osztály metódusát hívja.

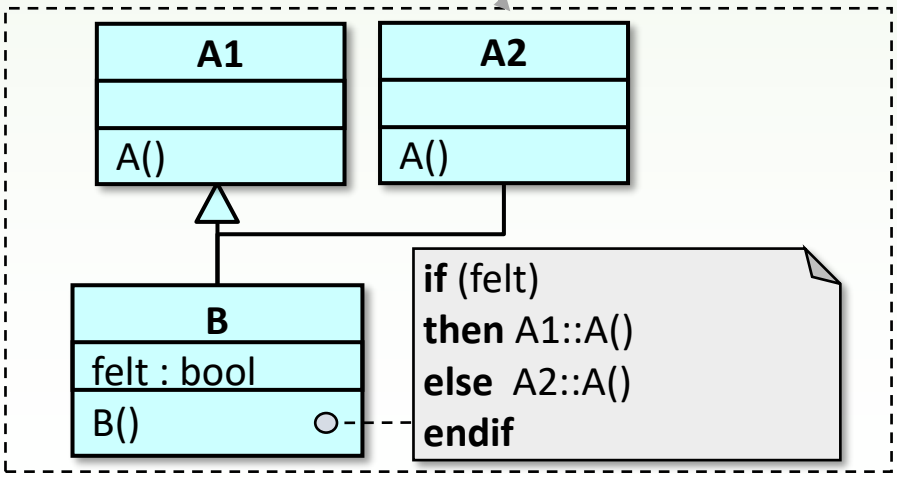


Felelősség átruházás rugalmassága

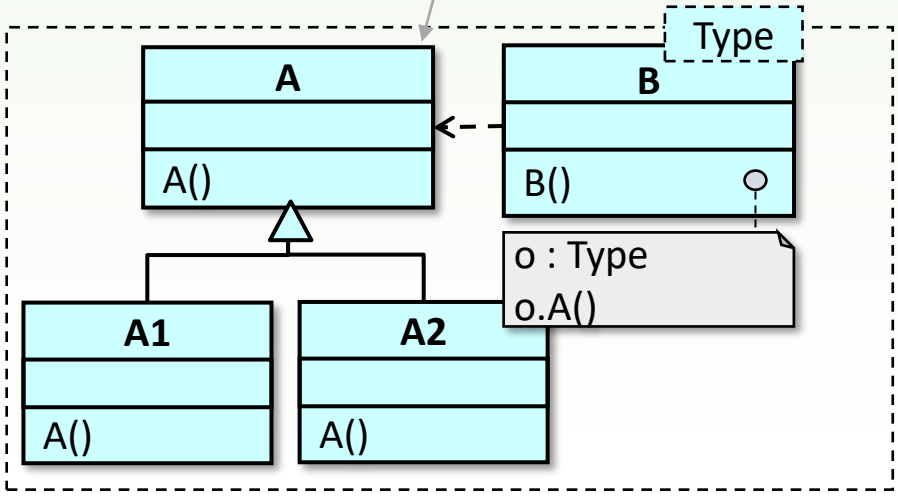
a B bármelyik objektumára beállítható futási időben, hogy az A1 vagy az A2 metodusA()-ját használja



több SOLID elvet is sért



fordítási időben kell a Type helyére A1 vagy A2 típust beírni



Halmaz osztály

Set
- repr : SetRepr
+ Set(n:int = 0)
+ SetEmpty() : void
+ Insert(int) : void
+ Remove(int): void
+ Select() : int {query}
+ Empty() : bool {query}
+ In(int) : bool {query}

```
#include "setrepr.h"
#include "array_set.h"
#include "sequence_set.h"

class Set {
public:
        Set(int n = 0) {
                if (0 == n) _repr = new SequenceSet;
                else        _repr = new ArraySet(n);
        }
        ~Set() { delete _repr; }
        void setEmpty()          { _repr->setEmpty(); }
        void insert(int e)       { _repr->insert(e); }
        void remove(int e)       { _repr->remove(e); }
        int    select()  const   { return _repr->select(); }
        bool empty()  const      { return _repr->empty(); }
        bool in(int e) const     { return _repr->in(e); }

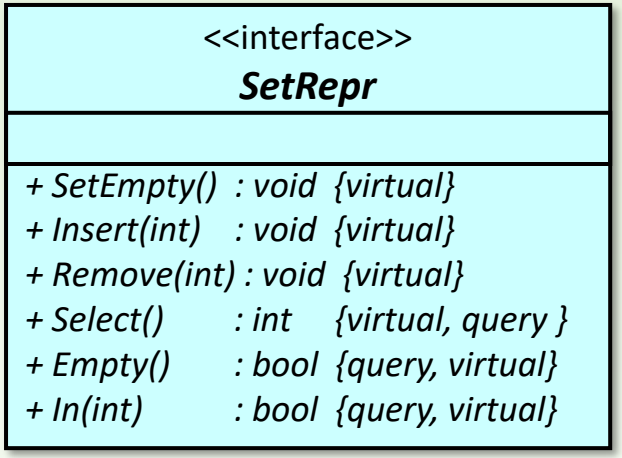
private:
        SetRepr *_repr;
};
```

inline definíciók

pointer

set.h

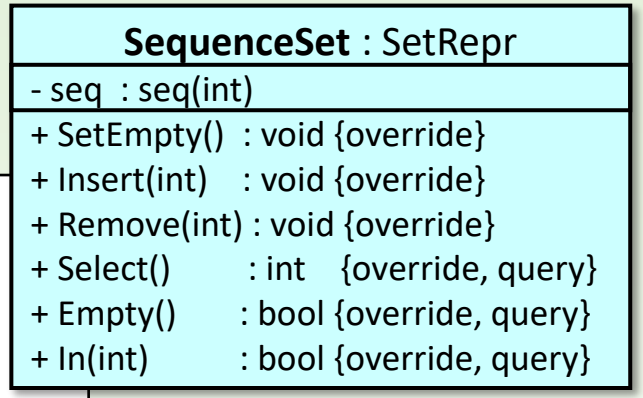
Reprezentáció interfésze




```
class SetRepr
{
public:
        virtual void setEmpty()         = 0;
        virtual void insert(int e)      = 0;
        virtual void remove(int e)      = 0;
        virtual int  select()  const    = 0;
        virtual bool empty() const      = 0;
        virtual bool in(int e) const    = 0;
        virtual ~SetRepr(){}
};
```

setrepr.h

Sorozat-reprezentáció




```
#include "setrepr.h"
#include <vector>

class SequenceSet : public SetRepr{
public:
        SequenceSet ():SetRepr() { setEmpty(); }
        void setEmpty()             override;
        void insert(int e)          override;
        void remove(int e)          override;
        int  select()    const      override;
        bool empty() const          override;
        bool in(int e) const        override;
private:
        std::vector<int> _seq;
        bool search(int e, unsigned int &ind) const;
};
```

segéd metódus

sequence_set.h

Sorozat-reprezentáció

SequenceSet : SetRepr

- seq : seq(int)

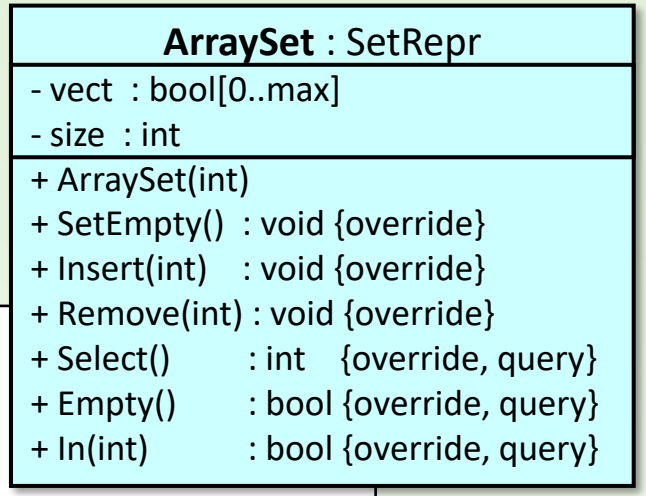
+ SetEmpty() : void {override}
+ Insert(int) : void {override}
+ Remove(int) : void {override}
+ Select() : int {override, query}
+ Empty() : bool {override, query}
+ In(int) : bool {override, query}

array_set.cpp

```
void SequenceSet::setEmpty() { _seq.clear(); }
void SequenceSet::insert(int e)
{
   unsigned int ind;
   if(!search(e,ind)) _seq.push_back(e);
}
void SequenceSet::remove(int e)
{
   unsigned int ind;
   if(search(e,ind)){
      _seq[ind] = _seq[_seq.size()-1];
      _seq.pop_back();
   }
}
int SequenceSet::select() const  { return _seq[0]; }
bool SequenceSet::empty() const
{
   return _seq.size()==0;
}
bool SequenceSet::in(int e) const
{
   unsigned int ind;
   return search(e,ind);
}
```

```
bool SequenceSet::search(int e,
                         unsigned int &ind) const
{
   bool l = false;
   for(unsigned int i=0; !l && i<_seq.size(); ++i ){
      l = _seq[i]==e; ind = i;
   }
   return l;
}
```

Tömb-reprezentáció




```
#include "setrepr.h"
#include <vector>

class ArraySet : public SetRepr{
public:
	ArraySet (int n) : SetRepr(), _vect(n+1), _size(0) {
		setEmpty();
	}
	void setEmpty()		override;
	void insert(int e)		override;
	void remove(int e)		override;
	int	select() const	override;
	bool empty() const	override;
	bool in(int e) const	override;
private:
	std::vector<bool> _vect;
	int _size;
};
```

array_set.h

Tömb-reprezentáció

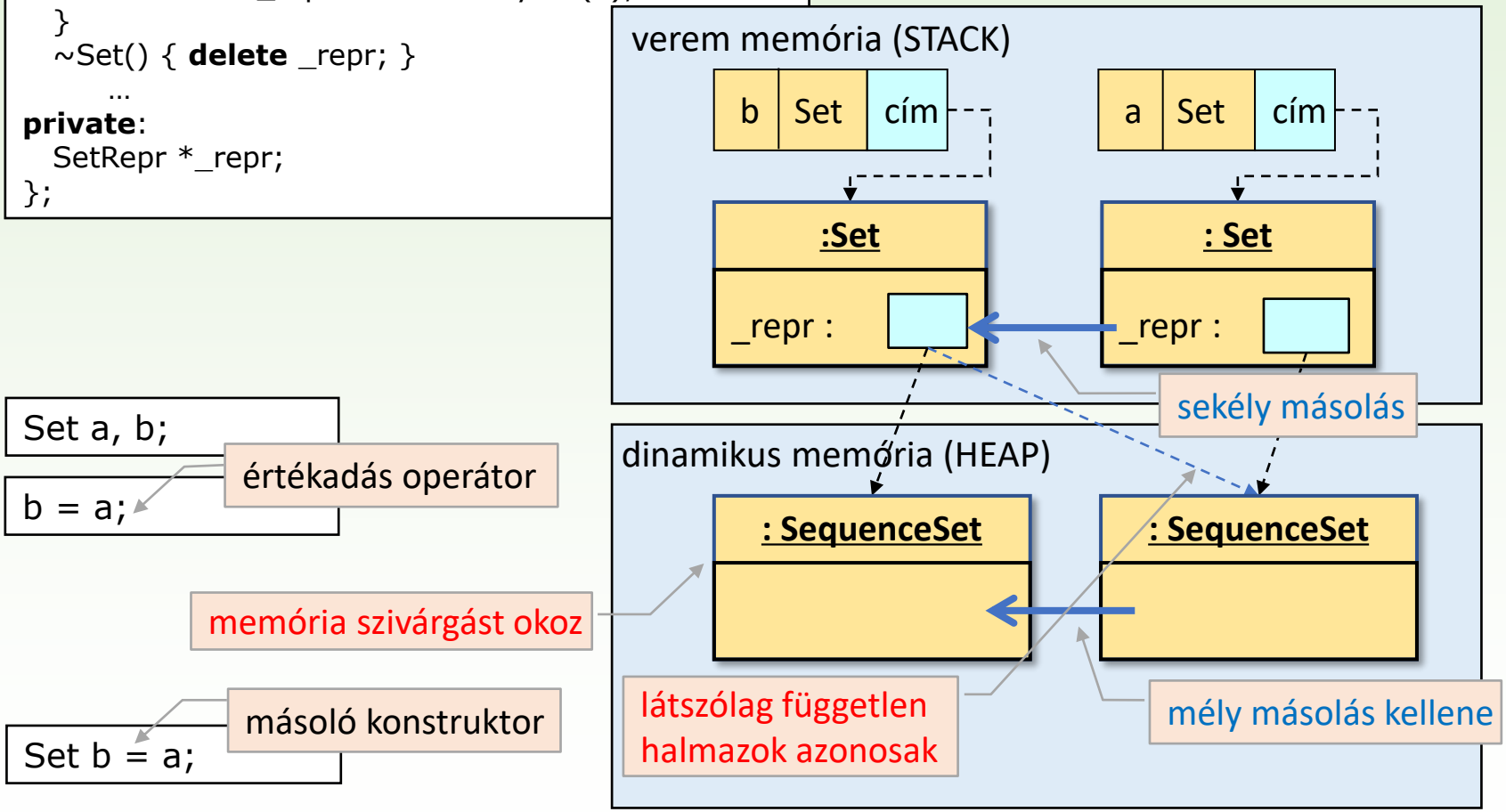
```
void ArraySet::setEmpty()
{
   for(unsigned int e=0; e<_vect.size(); ++e) _vect[e] = false;
   _size = 0;
}
void ArraySet::insert(int e)
{
   if(e<0 || e>int(_vect.size())-1) throw IllegalElementException(e);
   if(!_vect[e]){ _vect[e] = true; ++_size; }
}
void ArraySet::remove(int e)
{
   if(e<0 || e>int(_vect.size())-1) throw IllegalElementException(e);
   if(_vect[e]){  _vect[e] = false; --_size; }
}
int ArraySet::select() const
{
   int e;
   for(e=0; !_vect[e]; ++e);
   return e;
}
bool ArraySet:: empty() const {return _size==0; }
bool ArraySet::in(int e) const
{
   if(e<0 || e>int(_vect.size())-1) throw IllegalElementException(e);
   return _vect[e];
}
```

ArraySet : SetRepr
- vect : bool[0..max] - size : int
+ ArraySet(int) + SetEmpty() : void {override} + Insert(int) : void {override} + Remove(int) : void {override} + Select() : int {override, query} + Empty() : bool {override, query} + In(int) : bool {override, query}

array_set.cpp

Sekély- és mély másolás

```
class Set {
public:
  Set(int n = 0) {
    if (0 == n) _repr = new SequenceSet;
    else        _repr = new ArraySet(n);
  }
  ~Set() { delete _repr; }
      ...
private:
  SetRepr *_repr;
};
```



Másolás letiltása

```
#include "setrepr.h"
#include "array_set.h"
#include "sequence_set.h"

class Set {
public:
      Set(int n = 0) {
            if (0 == n) _repr = new SequenceSet;
            else        _repr = new ArraySet(n);
      }
      ~Set() { delete _repr; }
      void setEmpty()          { _repr->setEmpty(); }
      void insert(int e)       { _repr->insert(e); }
      void remove(int e)       { _repr->remove(e); }
      int  select() const      { return _repr->select(); }
      bool empty() const       { return _repr->empty(); }
      bool in(int e) const     { return _repr->in(e); }

private:
      SetRepr *_repr;

      Set(const Set& h);
      Set& operator=(const Set& h);
};
```

inline definíciók

pointer

Az alapértelmezett (implicite) másoló konstruktor és értékadás operátor nem végez mélymásolást. Tegyük explicite priváttá őket, hogy az esetleges nem tervezett használatuk fordítási hibát okozzon. Ekkor majd felülírjuk és publikussá tesszük őket.

Set
+ Set(n:int = 0) + setEmpty() : void + insert(int) : void + remove(int) : void + select() : int {query} + empty() : bool {query} + in(int) : bool {query}

set.h

Tervezési minták III.
2.rész
Halmaz felsorolása

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Egy alkalmazás

Keressünk egy természetes számokat tartalmazó halmazban olyan számot, amely nagyobb a halmaz legalább három másik eleménél!

(Ez a keresés biztos sikertelen lesz, ha nincs a halmazban legalább négy szám, és biztosan sikeres, ha van.)

- A feladat megoldható egy lineáris keresésbe ágyazott számlálással: az első olyan elemét keressük a halmaznak, amelyre teljesül, hogy a nálánál kisebb halmazbeli elemek száma nagyobb vagy egyenlő, mint 3. Mindkét programozási tételhez a halmaz elemeit kell felsorolni.

A = (h:set($\mathbb{N}$), l:$\mathbb{L}$, n:$\mathbb{N}$)

Ef = ($h = h_0$)

Uf = (l, n = $\text{SEARCH}_{e \in h_0}$ (kisebbekszáma(h_0, e) ≥ 3))

ahol kisebbekszáma(h_0, e) = $\sum_{\substack{u \in h_0 \\ e > u}} 1$

1. Próbálkozás

A halmaz műveletei alkalmasak a halmaz elemeinek felsorolása:

first()	~	–
current()	~	select()
next()	~	remove(select())
end()	~	empty()

```
bool l = false;
while ( !l && !h.empty() ){
        int e = h.select();
        int c = 0;
        while ( !h.empty() ) {
                int u = h.select();
                if (e > u) ++c;
                h.remove(u);
        }
        l = c>=3;
        h.remove(e);
}
```



Rossz megoldás:

- a felsorolás módosítja a gyűjteményt: a halmazt a belső felsorolás már első alkalommal kiüríti, így a külső ciklus „next”-je kivételt dob.
- a két felsorolás nem független: a belső ciklus folytatja, sőt be is fejezi a külső ciklusban elkezdett felsorolást

2. Próbálkozás

```
Set h;
// Beolvasás
…

Enumerator* enor1 = h.createEnumerator();
bool l = false;
for (enor1->first(); !l && !enor1->end(); enor1->next()){
        int n = enor1->current();
        int c = 0;
        Enumerator* enor2 = h.createEnumerator();
        for (enor2->first(); !enor2->end(); enor2->next()){
                if (n > enor2->current()) ++c;
        }
        l = c>=3;
}

if (l) cout << "A keresett szám: " << n << endl;
else   cout << "Nincs keresett szám.\n";
```

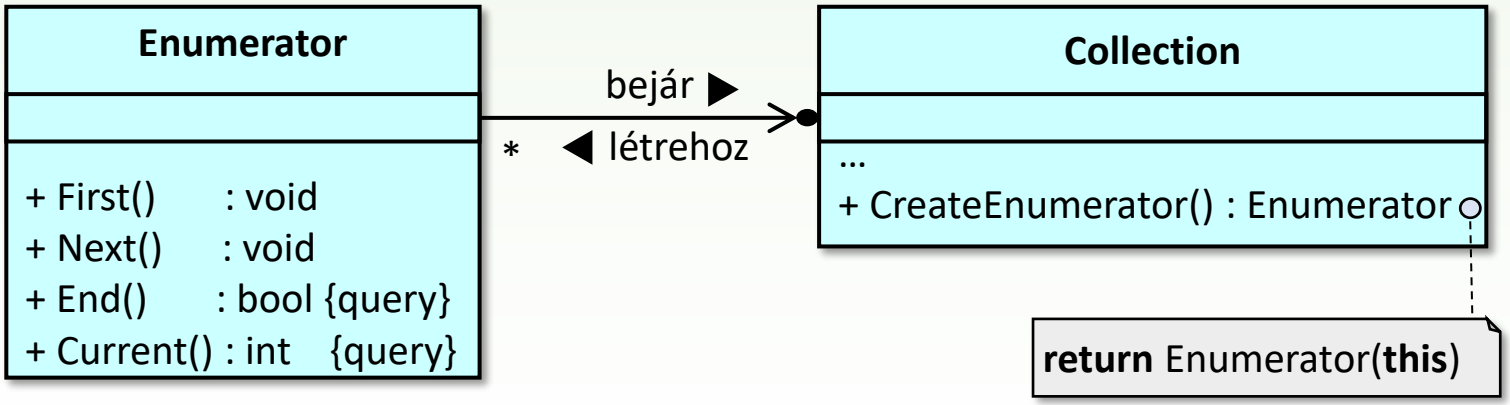
Külön felsoroló objektum legyen felelős a halmaz felsorolásáért, és mivel ilyet többet is létrehozhatunk, egymástól független felsorolások indíthatók.

Teendők:

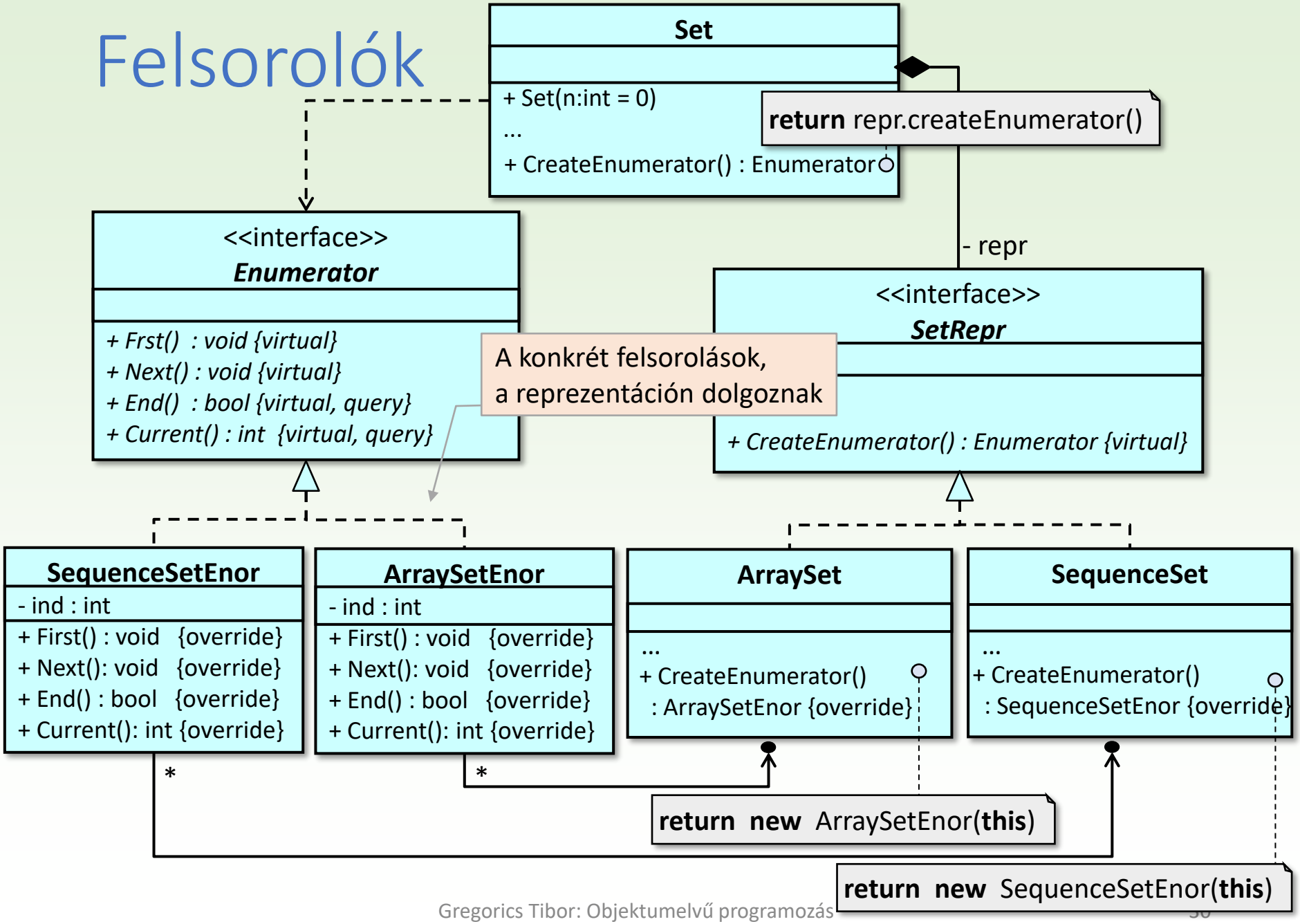
- Definiálni kell egy-egy felsoroló osztályt a kétféle halmaz-reprezentációhoz.
- Kiegészítjük a Set osztályt egy createEnumerator() metódussal, amelynek egy olyan felsorolót kell étrehoznia egy halmazhoz, amely a halmaz reprezentációjához illeszkedik.

Felsoroló (iterátor) tervezési minta

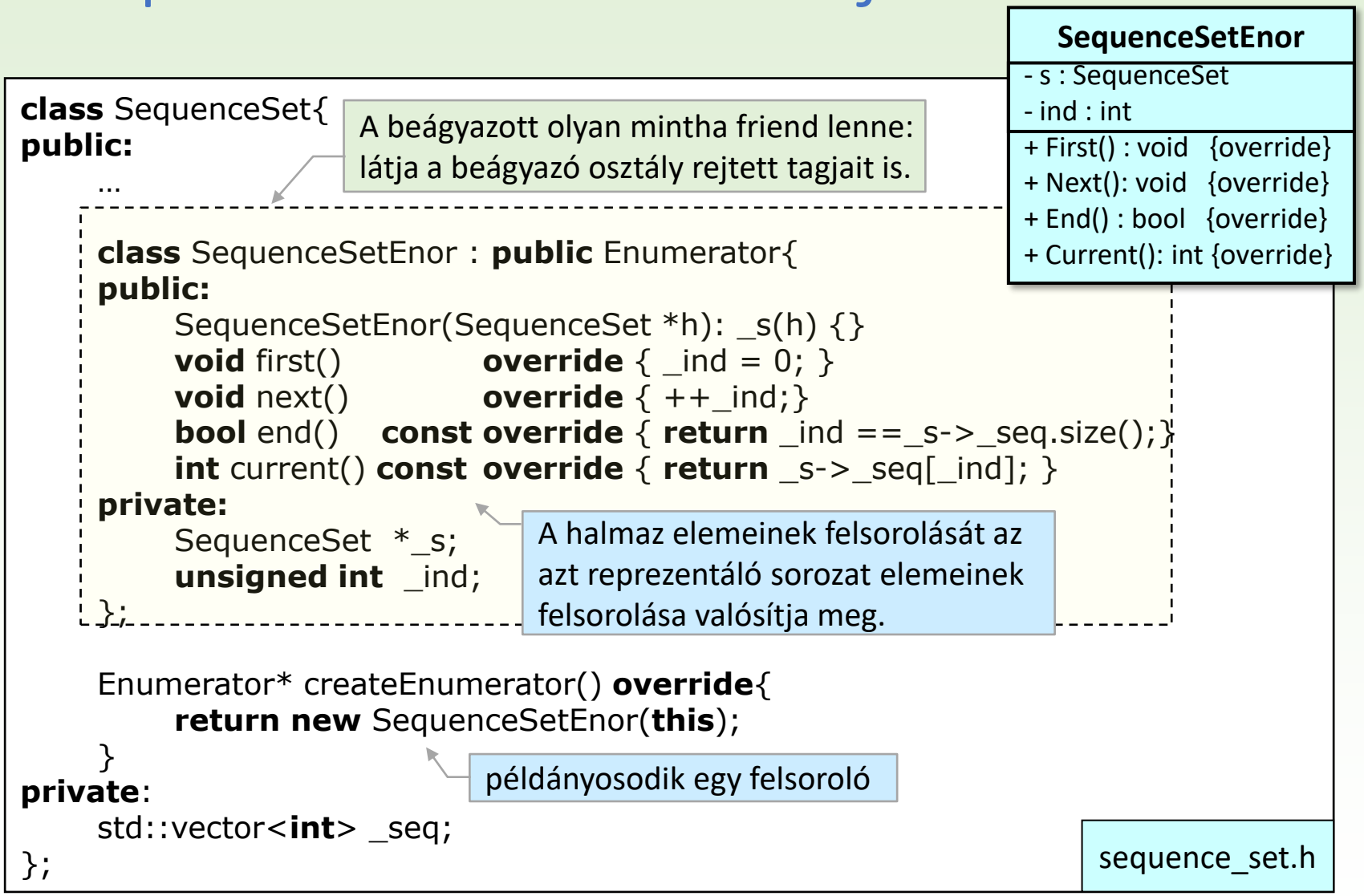
- Egy gyűjtemény elemeinek felsorolását (bejárását) egy attól független objektum (felsoroló) végzi, amely eléri a felsorolandó gyűjteményt (hivatkozik rá vagy annak konstans másolatára). A felsoroló objektumot a gyűjtemény hozza létre.



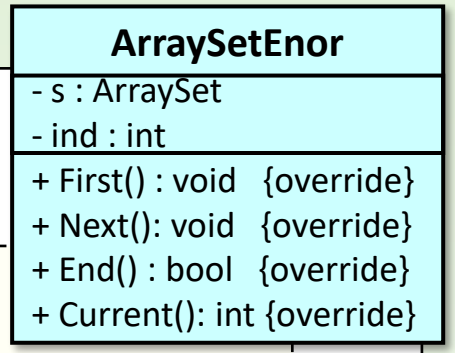
Felsorolók


Set
+ Set(n:int = 0)
...
+ CreateEnumerator() : Enumerator
return repr.createEnumerator()
- repr
<<interface>>
Enumerator
+ Frst() : void {virtual}
+ Next() : void {virtual}
+ End() : bool {virtual, query}
+ Current() : int {virtual, query}
A konkrét felsorolások,
a reprezentáción dolgoznak
<<interface>>
SetRepr
+ CreateEnumerator() : Enumerator {virtual}
SequenceSetEnor
- ind : int
+ First() : void {override}
+ Next(): void {override}
+ End() : bool {override}
+ Current(): int {override}
ArraySetEnor
- ind : int
+ First() : void {override}
+ Next(): void {override}
+ End() : bool {override}
+ Current(): int {override}
ArraySet
...
+ CreateEnumerator()
: ArraySetEnor {override}
SequenceSet
...
+ CreateEnumerator()
: SequenceSetEnor {override}
*
*
return new ArraySetEnor(this)
return new SequenceSetEnor(this)

SequenceSet felsorolója



ArraySet felsorolója




```
class ArraySet{
public:
    …
    class ArraySetEnor : public Enumerator{
    public:
        ArraySetEnor(ArraySet *h): _s(h) {}
        void first()    override { _ind = -1; next(); }
        void next()     override {
            for (++_ind;_ind<_s->_vect.size() && !_s->_vect[_ind]; ++_ind);
        }
        bool end()      const override {return _ind==_s->_vect.size();}
        int  current()  const override {return _ind; }
    private:
        ArraySet  *_s;
        unsigned int _ind
    };

    Enumerator* createEnumerator() override{
        return new ArraySetEnor(this);
    }
private:
    std::vector<bool> _vect;
    int _size;
};
```

A halmaz elemeinek felsorolását az azt reprezentáló tömb true értékeihez tartozó indexek felsorolása valósítja meg.

példányosodik egy felsoroló

array_set.h

Javítás

Tegyük biztonságossá a halmaz felsorolását!

- Probléma: ha a felsorlás közben megváltozik a halmaz (pl. a setEmpty(), insert(), remove(), a halmaznak halmazt adunk értékül), akkor a felsorolás elromolhat.
- Megoldás: Zárjuk ki az ilyen műveleteket végrehajtását felsorolás közben.

```
Set h;
 ...
Enumerator * enor = h.createEnumerator();
for (enor->first(); !enor->end(); enor->next()){
      h.remove(enor->current());
}
```

Hibát okozhat a felsorolás során végrehajtott törlés, hiszen elveszítjük a felsorolás aktuális elemét.



Kizárás megvalósítása

Kritikus műveletet ne hajtsunk végre, de dobjunk kivételt, ha felsoroló dolgozik halmazon.

```
Set h;
try {
        ...
        h.remove(enor->current());
        ...
} catch(std::exception &ex){
        cout << ex.what() << endl;
}
```

```
class SetRepr {
public:
        SetRepr(): _enumeratorCount(0){}
        ...
        int getEnumCount() const { return _enumeratorCount; }
protected:
        int _enumeratorCount;
};
```

már nem interfész, de még absztrakt osztály

aktív felsorolók száma

setrepr.h

```
class UnderTraversalException : public std::exception {
public:
        const char* what() const noexcept override {
                return "Under traversal";
        }
};
```

setrepr.h

```
class Set {
public:
        ...
        void Set::remove(int e)
        {
                if (_repr->getEnumCount()!=0)
                        throw UnderTraversalException();
                _ref->remove(e);
        }
        ...
};
```

ismerni kell a halmazon dolgozó felsorolók számát

set.h

Felsorolók számának nyilvántartása a SequenceSet-ben

```
class SequenceSet : public SetRepr{
public:
    SequenceSet() : SetRepr() { setEmpty(); }
    ...
    class SequenceSetEnor : public Enumerator{
    public:
        SequenceSetEnor(SequenceSet *h): _s(h)
        { ++(_s->_enumeratorCount); }
        ~ SequenceSetEnor() { --(_s->_enumeratorCount); }
        ...
    };

    Enumerator* createEnumerator() override {
        return new SequenceSetEnor(this);
    }
};
```

lenullázza a felsorolók számlálóját

Amikor egy új felsorolót példányosítunk egy SequenceSet objektumhoz, akkor annak felsoroló-számlálóját növeljük.

A felsoroló megszűnésekor a felsoroló-számlálót csökkentjük.

sequence_set.h

Felsorolók számának nyilvántartása az ArraySet-ben

```
class ArraySet : public SetRepr {
public:
    ArraySet(int n) : SetRepr(), _vect(n+1), _size(0) {
        setEmpty();
    }
    ...
    class ArraySetEnor : public Enumerator {
    public:
        ArraySetEnor(ArraySet *h): _s(h)
        { ++(_s->_enumeratorCount); }
        ~ArraySetEnor() { --(_s->_enumeratorCount); }
        ...
    };

    Enumerator * createEnumerator() override {
        return new ArraySetEnor(this);
    }
};
```

lenullázza a felsorolók számlálóját

Amikor egy új felsorolót példányosítunk egy ArraySet objektumhoz, akkor annak felsoroló-számlálóját megnöveljük.

a felsoroló megszűnésekor a felsoroló-számlálót csökkentjük

array_set.h

Tervezési minták III.

3.rész

Halmaz sablon

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Általánosítás

- Olyan halmaz példányosítását is szeretnénk támogatni, amelyben nem természetes számokat, hanem más típusú elemeket lehet tárolni. Ezért generikus osztályokká alakítjuk át az osztályainkat.
- Vegyük azonban figyelembe, hogy a tömbös reprezentációval csak olyan halmazokat tudunk ábrázolni, amelyekbe felső korláttal rendelkező természetes számokat lehet elhelyezni, azaz ilyenkor int lehet csak a sablonparaméter.

fordítási időben osztályként példányosodik az osztálysablon
futási időben objektumként példányosodik az osztály

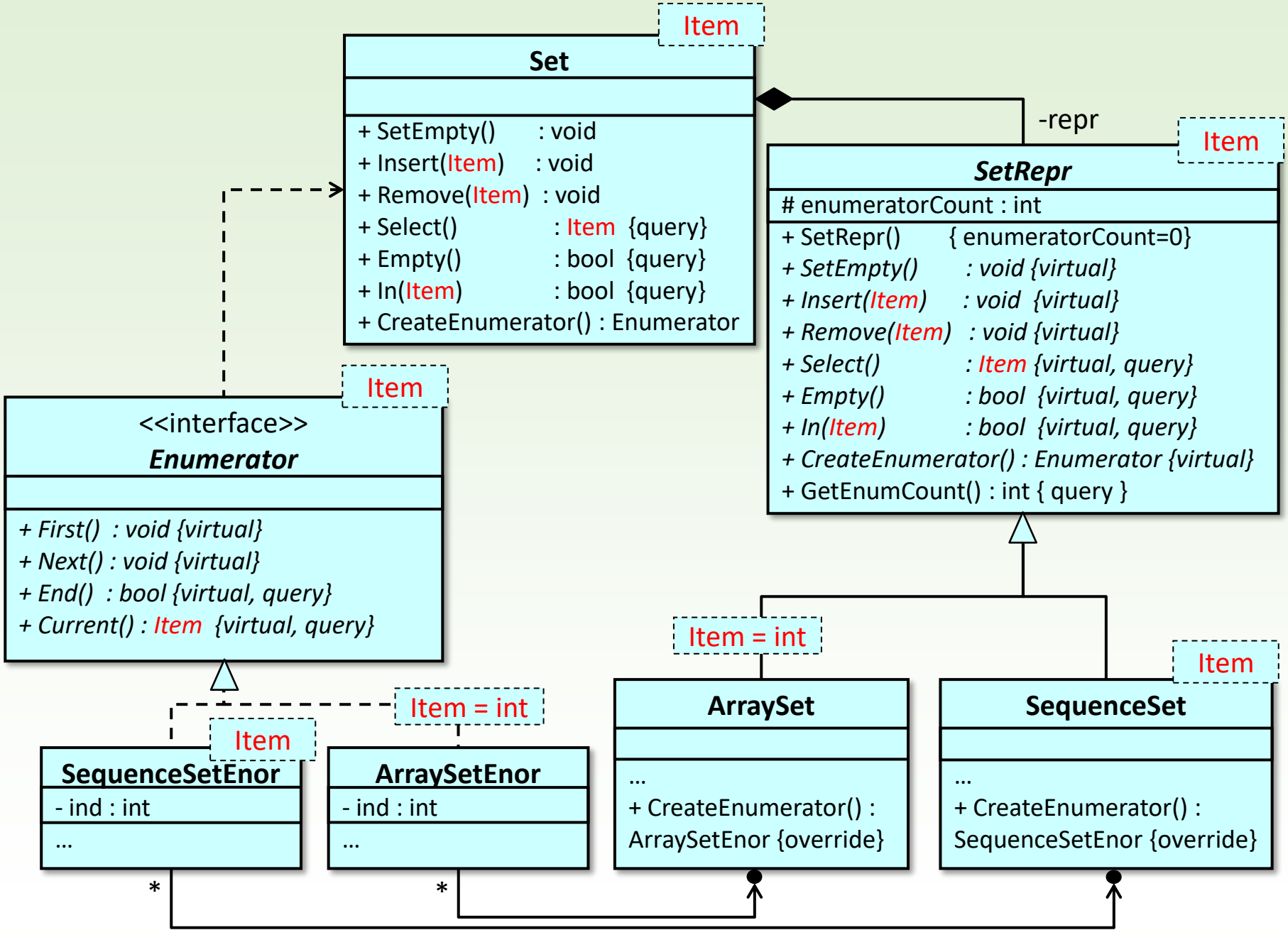
```
Set<int>    h1(100);
Set<int>    h2;
Set<string> h3;

h1.insert(42);
h2.insert(1456);
h3.insert("alma");

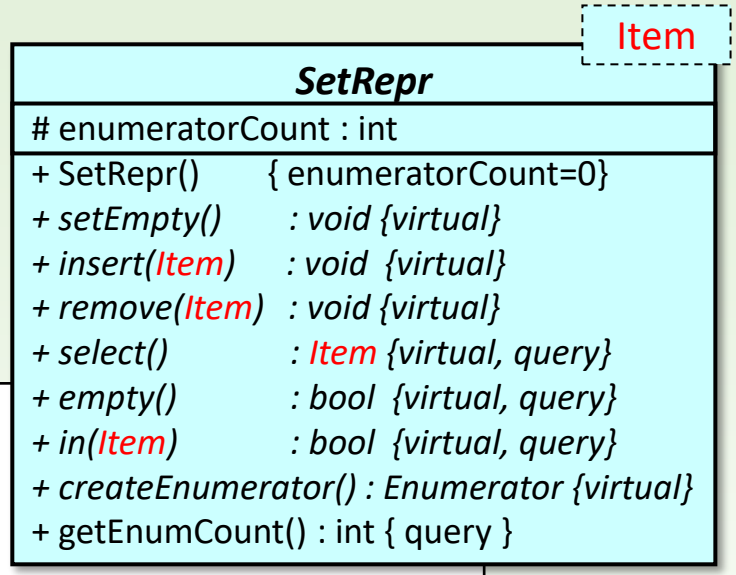
Enumerator<int>    *enor1 = h1.createEnumerator();
Enumerator<string> *enor2 = h3.createEnumerator();
```

A felsoroló osztályok is osztálysablonok példányai lesznek, amelyek típusparaméterét egyeztetni kell a halmazéval.

main.cpp


Item
Set
+ SetEmpty() : void
+ Insert(Item) : void
+ Remove(Item) : void
+ Select() : Item {query}
+ Empty() : bool {query}
+ In(Item) : bool {query}
+ CreateEnumerator() : Enumerator
-repr
Item
SetRepr
enumeratorCount : int
+ SetRepr() { enumeratorCount=0}
+ SetEmpty() : void {virtual}
+ Insert(Item) : void {virtual}
+ Remove(Item) : void {virtual}
+ Select() : Item {virtual, query}
+ Empty() : bool {virtual, query}
+ In(Item) : bool {virtual, query}
+ CreateEnumerator() : Enumerator {virtual}
+ GetEnumCount() : int { query }
Item
<<interface>>
Enumerator
+ First() : void {virtual}
+ Next() : void {virtual}
+ End() : bool {virtual, query}
+ Current() : Item {virtual, query}
Item = int
Item
SequenceSetEnor
- ind : int
...
Item = int
ArraySetEnor
- ind : int
...
ArraySet
...
+ CreateEnumerator() : ArraySetEnor {override}
Item
SequenceSet
...
+ CreateEnumerator() : SequenceSetEnor {override}
*
*

SetRepr osztály-sablonja



jelöli a sablont, megadja a sablon paramétereit

```
template <typename Item>
class SetRepr {
public:
        SetRepr() : _enumeratorCount(0) {}
        virtual ~SetRepr(){};
        virtual void setEmpty()         = 0;
        virtual void insert(Item e)     = 0;
        virtual void remove(Item e)     = 0;
        virtual Item select()     const = 0;
        virtual bool empty()      const = 0;
        virtual bool in(Item e)const = 0;
        virtual Enumerator<Item>* createEnumerator() = 0;
        int getEnumCount() const { return _enumeratorCount; }
protected:
        int _enumeratorCount;
};
```

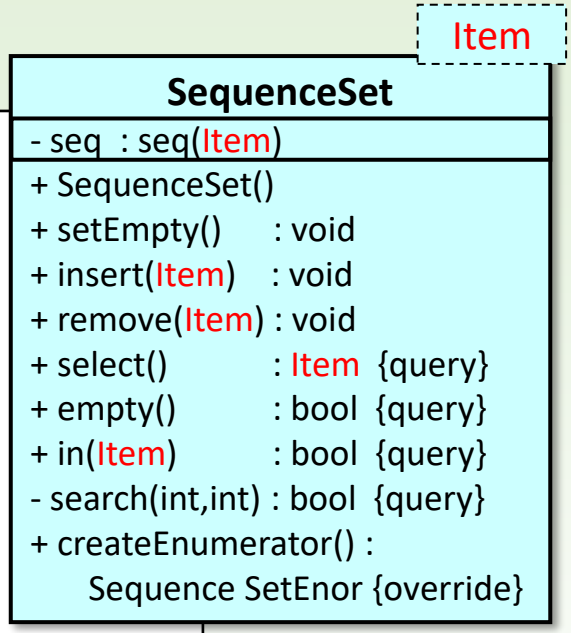
Miért .hpp?

setrepr.hpp

SequenceSet osztály-sablonja

```
template <typename Item>
class SequenceSet : public SetRepr<Item>{
public:
        SequenceSet () : SetRepr<Item> { setEmpty(); }
        void setEmpty()        override;
        void insert(Item e)    override;
        void remove(Item e)  override;
        Item select()    const override;
        bool empty()   const override;
        bool in(Item e)const override;
        …
private:
        std::vector<Item> _seq;
        bool search(Item e, unsigned int &ind) const;
};
…
```

sequence_set.hpp



.h és .cpp helyett: közös állományba kerül az osztálysablon és a sablon-metódusainak definíciója

SequenceSet metódus-sablonjai

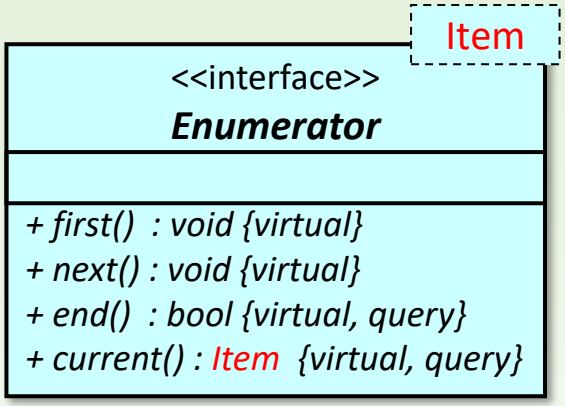
egy osztálysablon metódusai függvénysablonok

```
template <typename Item>
void SequenceSet<Item>::insert(int e)
{
      unsigned int ind;
      if (!search(e,ind)) _seq.push_back(e);
}
template <typename Item>
void SequenceSet<Item>::remove(int e)
{
      unsigned int ind;
      if (search(e,ind)){
            _seq[ind] = _seq[_seq.size()-1];
            _seq.pop_back();
      }
}
template <typename Item>
int SequenceSet<Item>::select() const
{
      return _seq[0];
}

...
```

sequence_set.hpp

Enumerator interfész-sablon




```
template <typename Item>
class Enumerator {
public:
        virtual void first()              = 0;
        virtual void next()               = 0;
        virtual bool end()      const     = 0;
        virtual Item current()  const     = 0;
        virtual ~Enumerator(){};
};
```

enumerator.hpp

SequenceSetEnor osztály-sablonja

a beágyazás miatt ez is egy Item típus-paraméterű sablon, de nem kell kiírni újra, hogy ez egy template

```
template <typename Item>
class SequenceSet : public SetRepr<Item>{
public:
    SequenceSet () : SetRepr<Item> { setEmpty(); }
    ...
    class SequenceSetEnor : public Enumerator<Item>{
    public:
        SequenceSetEnor(SequenceSet<Item> *h): _s(h) {}
        void first()              override { _ind = 0; }
        void next()               override { ++_ind;}
        bool end()      const override { return _ind ==_s->_seq.size();}
        Item current() const override { return _s->_seq[_ind]; }
    private:
        SequenceSet<Item>  *_s;
        unsigned int  _ind;
    };

    Enumerator<Item>* createEnumerator() override{
        return new SequenceSetEnor<Item> (this);
    }
};
```

Item

SequenceSetEnor

- s : SequenceSet
- ind : int

+ first() : void {override}
+ next() : void {override}
+ end() : bool {override}
+ current() : Item {override}

sequence_set.hpp

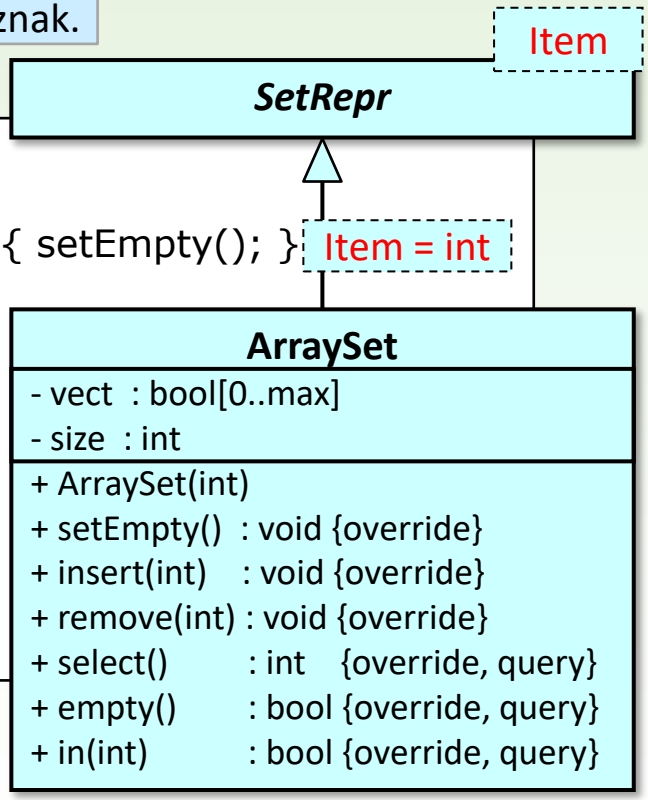
ArraySet és ArraySetEnor osztály

Sem az ArraySet, sem az ArraySetEnor nem lesz sablon, és a SetRepr helyett a SetRepr<int>-ből, illetve az Enumerator helyett az Enumerator<int>ből származnak.

```
class ArraySet : public SetRepr<int>{
public:
 ArraySet (int n): SetRepr<int>(), _vect(n+1), _size(0) { setEmpty(); }
    ...
    class ArraySetEnor : public Enumerator<int>{
          ...
    };

    Enumerator<int>* createEnumerator() override{
          return new ArraySetEnor(this);
    }
};
```

array_set.hpp

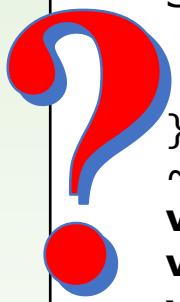


Set osztály-sablon

```
template <typename Item>
class Set {
public:
    Set(int n = 0) {
        if (0 == n) _repr = new SequenceSet<Item>;
        else        _repr = new ArraySet(n);
    }
    ~Set() { delete _repr; }
    void setEmpty()      { _repr->setEmpty(); }
    void insert(Item e)  { _repr->insert(e); }
    void remove(Item e) { _repr->remove(e); }
    Item select()  const  { return _repr->select(); }
    bool empty()   const { return _repr->empty(); }
    bool in(Item e)const { return _repr->in(e); }

    Enumerator<Item>* createEnumerator() { _repr->createEnumerator(); }
private:
    SetRepr<Item> *_repr;

    Set(const Set& h) ;
    Set& operator=(const Set& h);
};
```



Fordítási hiba:
Ha Item (nem az int), hanem pl. a string, akkor ez az értékadás hibás (típus konfliktus), hiszen ArraySet a SetRepr<int> alosztálya, de a _repr SetRepr<string> típusú.

Item

Set

+ Set(n:int = 0)
+ setEmpty() : void
+ insert(Item) : void
+ remove(Item) : void
+ select() : Item {query}
+ empty() : bool {query}
+ in(Item) : bool {query}

Viszont erre az ágra csak Item=int esetén lenne szükség. Csak ekkor kellene lefordítani.

set.hpp

Sablon specializáció

- Lehetőség van egy osztály-sablon, vagy akár csak egy függvény-sablon alternatív definícióira a sablonparaméter értékétől függően.

Két gyártófüggvény-sablont definiálunk: az általános mellett arra a speciális esetre, amikor Item=int.

```
template <typename Item>
class Set {
public:
        Set(int n = 0) { _repr = createSetRepr<Item>(n); }
        …
private:
        SetRepr<Item>* _repr;

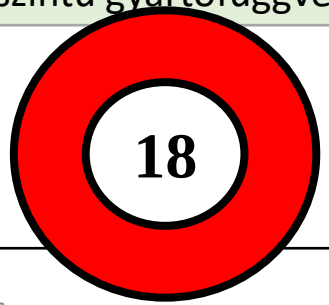
        static SetRepr<Item>* createSetRepr(int n) {
                return new SequenceSet<Item>;
        }
};
```

általános osztályszintű gyártófüggvény-sablon

set.hpp

speciális osztályszintű gyártófüggvény-sablon

```
template<>
SetRepr<int>* Set<int>::createSetRepr(int n) {
        if (0 == n) return new SequenceSet<int>;
        else        return new ArraySet(n);
}
```



set.hpp